Experiment: 1 **Singly Linked List**

Aim:

To write and execute a C program to implement a **Singly Linked List using dynamic memory allocation** and perform insertion, deletion, searching, and display operations.

Algorithm:

1. Append

1. Create a new node using malloc().
2. Store data and set newnode->next = NULL.
3. If head == NULL, assign head = newnode.
4. Otherwise, traverse from head until temp->next == NULL.
5. Set temp->next = newnode.
6. Display the appended element.

2. Prepend

1. Create a new node dynamically.
2. Store data and set newnode->next = NULL.
3. If the list is empty, assign head = newnode.
4. Otherwise: assign: newnode->next = head
   head = newnode
5. Display the prepended element.

3. Display

1. Set temp = head.
2. Repeat until temp != NULL:
   Display temp->data
   temp = temp->next
3. Stop when temp becomes NULL.

4. Insert at a Given Position

1. Create a new node and store data.
2. If pos == 0: newnode->next = head and head = newnode
3. Otherwise, initialize curr = head, prev = NULL.
4. Traverse the list until the required position is reached.
5. Link the new node between prev and curr:
   newnode->next = curr
   prev->next = newnode
6. Display the inserted element and position.

5. Search

1. Set temp = head, found = 0, and pos = 1.
2. Traverse the list until temp != NULL.
   Compare temp->data with the search element.
   If matched: found = 1 break
   Otherwise, move to the next node and increment pos.
3. If found == 1, display the position; otherwise display **element not found**.

6. Delete

1. Set temp = head, prev = NULL, and found = 0.
2. Traverse the list until the required element is found.
3. If the element is not found, display **element not found**.
4. If the node is the first node: head = temp->next
5. Else if it is the last node: prev->next = NULL
6. Otherwise: prev->next = temp->next
7. Display the deleted element and release its memory using: free(temp)

Program:

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
   int data;
   struct node *next;
};
struct node *head, *newnode, *curr, *prev, *temp;

void prepend(int data)
{
   newnode = (struct node*)malloc(sizeof(struct node));
   newnode->data = data;
   newnode->next = NULL;

   if(head == NULL)
   {
      head = newnode;
   }
   else
   {
      newnode->next = head;
      head = newnode;
   }
   printf("data prepended:%d \n",data);

}

void append(int data)
{
   newnode = (struct node*)malloc(sizeof(struct node));
   newnode->data = data;
   newnode->next = NULL;
```

```
   if(head == NULL)
   {
      head = newnode;
   }
   else
   {
      struct node *temp = head;
      while(temp->next != NULL)
      {
         temp = temp->next;
      }
      temp->next = newnode;
   }
   printf("data appended:%d \n",data);
}

void display()
{
   temp = head;
   while(temp != NULL)
   {
      printf(" %d -> ",temp->data);
      temp = temp->next;
   }
}
void insert(int data,int pos)
{
   newnode = (struct node*)malloc(sizeof(struct node));
   newnode->data = data;
   newnode->next = NULL;

   if(pos == 0)
   {
      newnode->next = head;
      head = newnode;
   }
   else
   {
      int i = 0;
      curr = head;
      prev = NULL;
      while(curr != NULL)
      {
         if(i == pos)
         {
            break;
         }
```

```
            prev = curr;
            curr = curr->next;
            i++;
        }
        newnode->next = curr;
        prev->next = newnode;
    }
    printf("data inserted:%d at position:%d\n",data,pos);
}
void search(int data)
{
    temp = head;
    int found = 0, pos = 1;
    while(temp != NULL)
    {
        if(temp->data == data)
        {
            found = 1;
            break;
        }
        temp = temp->next;
        pos++;
    }
    if(found == 0)
    {
        printf("data not found");
    }
    else
    {
        printf("data found @:%d",pos);
    }
}
void delete(int data)
{
    int found = 0;
    prev = NULL;
    temp = head;
    while(temp != NULL)
    {
        if(temp->data == data)
        {
            found = 1;
            break;
        }

        prev = temp;
        temp = temp->next;
    }
```

```
    if(found == 0)
    {
       printf("\n element not found");
    }
    else
    {
       if(temp == head)
       {
          head = temp->next;
       }
       else if(temp->next == NULL)
       {
          prev->next = NULL;
       }
       else
       {
          prev->next = temp->next;
       }
       printf("\nelement delted:%d\n",temp->data);
       free(temp);
    }
}
int main()
{
    append(10);
    append(20);
    append(30);
    display();
    printf("\n");
    prepend(40);
    display();
    delete(20);
    printf("\n");
    display();
    printf("\n");
    insert(50,0);
    display();
    printf("\n search element: 40 \n");
    search(40);
    printf("list data\n ");
    display();
    return 0;
}
```

Output:

```
data appended:10
data appended:20
data appended:30
 10 -> 20 -> 30 ->
data prepended:40
 40 -> 10 -> 20 -> 30 ->
element delted:20
 40 -> 10 -> 30 ->
data inserted:50 at position:0
 50 -> 40 -> 10 -> 30 ->
 search element: 40
data found @:2list data
 50 -> 40 -> 10 -> 30 ->
```

Result:

Thus, the Singly Linked List was successfully implemented in C, and the operations append, prepend, insert, delete, search, and display were successfully performed.

Experiment :2 **Circular Linked List**

Aim:

To write and execute a C program to implement a **Circular Linked List using dynamic memory allocation** and perform **insertion, deletion, and display operations**.

Algorithm:

1. Create Node

1. Allocate memory dynamically for a new node using malloc().
2. Store the given data in newnode->data.
3. Initialize newnode->next to NULL.
   Create new node
   Store data: Set newnode->next = NULL

2. Append

1. Create a new node.
2. If the list is empty:
   head = tail = newnode
   tail->next = head
3. Otherwise:
   tail->next = newnode
   tail = newnode
   tail->next = head
4. Display the appended element

3. Prepend

1. Create a new node.
2. If the list is empty:
   head = tail = newnode
   tail->next = head
3. Otherwise:
   newnode->next = head
   head = newnode
   tail->next = head
4. Display the prepended element.

4. Insert

1. Set temp = head.
2. Traverse the circular list.
   Compare temp->data == key, if key is found then break
   temp = temp->next
   Continue Until temp != head
3. If the key is not found, display **key element not found**.
4. If the key is found, create a new node.
5. If the key node is the tail:
   Set temp->next = newnode

tail = newnode
tail->next = head

Otherwise:

newnode->next = temp->next
temp->next = newnode

6. Display the inserted element.

5. Display

1. Set temp = head.
2. traverse the circular list.

Display temp->data.
Move to the next node:
temp = temp-next

3. Continue until when temp != head.

6. Delete an Element

1. Set temp = head.
2. Traverse the circular list.

Compare temp->data == key, if key is found then break
temp = temp->next
Continue Until temp != head

3. If the key is not found, display **key element not found**.
4. If the key is found,

If the node to be deleted is head:

head = head->next
tail->next = head

5. Display the deleted element.

Program:

```
#include <stdio.h>
#include <malloc.h>
struct node
{
   int data;
   struct node *next;
};
struct node *head, *tail, *newnode, *temp, *prev;
void createnode(int d)
{
   newnode = (struct node*)malloc(sizeof(struct node));
   newnode->data = d;
   newnode->next = NULL;
}
```

```
void insert(int key, int data)
{
   int found = 0;
   temp = head;
   do
   {
      if(temp->data == key)
      {
         found = 1;
         break;
      }
      temp = temp->next;
   }while(temp != head);
   if(found == 0)
   {
      printf("key element not found \n");
   }
   else
   {
      createnode(data);
      if(temp == tail)
      {
         temp->next = newnode;
         tail = newnode;
         tail->next = head;
      } else {
         newnode->next = temp->next;
         temp->next = newnode;
      }
      printf("inserted :%d after :%d",data,key);
   }
}
void prepend(int d)
{
   createnode(d);
   if(head == NULL)
   {
      head = tail = newnode;
      tail->next = head;
   }
   else
   {
      newnode->next = head;
      head = newnode;
      tail->next = head;
   }
   printf("data prepended :%d \n",d);
}
```

```
void append(int d)
{
   createnode(d);
   if(head == NULL)
   {
      head = tail = newnode;
      tail->next = head;
   }
   else
   {
      tail->next = newnode;
      tail = newnode;
      tail->next = head;
   }
   printf("data appended :%d \n",d);
}

void display()
{
   temp = head;
   do
   {
      printf("%d ->",temp->data);
      temp = temp->next;

   }while(temp != head);
}

void delete(int data)
{
   int found = 0;
   temp = head;
   do  {
      if(temp->data == data)    {
         found = 1;
         break;
      }
      temp = temp->next;
   }while(temp != head);

   if(found == 0)
   {
      printf("key element not found \n");
   }
   else
   {
```

```
        if(temp == head)
        {
           head = head->next;
           tail->next = head;
           free(temp);
        }

        printf("data deleted :%d \n",data);
    }
}

int main()
{
    append(10);
    append(20);
    append(30);
    prepend(40);
    display();
    printf("\n--\n");
    insert(20,25);
    display();
    printf("\n--\n");
    delete(10);
    display();
    printf("\n--\n");
    display();
}
```

Output:

```
data appended :10
data appended :20
data appended :30
data prepended :40
40 ->10 ->20 ->30 ->
--
inserted :25 after :20
40 ->10 ->20 ->25 ->30 ->
--
data deleted :10
40 ->20 ->25 ->30 ->
40 ->20 ->25 ->30 ->
```

Result:

Thus, the Circular Linked List was successfully implemented in C and the operations append, prepend, insertion after a specified element, deletion, and display were executed successfully.

Experiment: 3 **Doubly Linked List**

Aim:
To write and execute a C program to implement a Doubly Linked List and perform operations insertion, deletion, forward traversal, reverse traversal, and count the number of elements.

Algorithm:

1. Create and Initialize Node

1. Define a node containing:
    - data – stores the element.
    - prev – stores the address of the previous node.
    - next – stores the address of the next node.
2. Allocate memory dynamically using malloc().
3. Store the given data in the node.
4. Set prev and next to NULL.

2. Append an Element

1. Create a new node.
2. If head == NULL:
   assign the new node to head.
   Otherwise:
   traverse from head until the last node.
   Link the new node with the last node:
   temp->next = newnode
   newnode->prev = temp
   Update tail to the new node.
3. Increment total_elements.

3. Prepend an Element

1. Create a new node.
2. If the list is empty:
   assign the new node to head.
3. Otherwise:
   newnode->next = head
   head->prev = newnode
   head = newnode
   Increment total_elements.

4. Delete an Element

1. Set curr = head and before = NULL.
2. Traverse the list until the required element is found.
3. If the element is not found, display **element not found**.
4. If the node is the first node:
   head = curr->next
   head->prev = NULL

5. If the node is the last node:
   before->next = NULL
   tail = before
   Otherwise, connect the previous and next nodes:
   before->next = curr->next
   curr->next->prev = before
   Decrement total_elements.

5. Display the List

1. Set temp = head.
2. Traverse the list until temp != NULL.
   Display temp->data.
   Move to the next node: temp = temp->next

6. Reverse Display

1. Set temp = tail.
2. Traverse backwards untile temp != NULL.
   Display temp->data.
   Move to the previous node: temp = temp->prev

Program:

```
#include<stdio.h>
#include<malloc.h>
struct node
{
  int data;
  struct node *prev,*next;
};
struct node *head,*newnode,*temp,*tail,*before,*curr;
int total_elements = 0;

void delete(int data)
{
  int found = 0;
  before = NULL;
  curr = head;
  while(curr != NULL)    {
    if(curr->data == data)
    {
      found = 1;
      break;
    }
    before = curr;
    curr = curr->next;
  }
  if(found == 0)
  {
    printf("\n element not found \n");
```

```
    }
    else
    {
        if(curr == head)
        {
            head = curr->next;
            head->prev = NULL;
            free(curr);
        }
        else if(curr->next == NULL)        {
            before->next = NULL;
            tail = before;
            free(curr);
        }  else {
            before->next = curr->next;
            curr->next->prev = before;
            free(curr);
        }
        total_elements--;
       printf("the element deleted: %d \n",data);
    }
}
void prepend(int data)
{
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;
    newnode->next = newnode->prev = NULL;

    if(head == NULL)     {
        head = newnode;
    } else  {
        newnode->next = head;
        head->prev = newnode;
        head = newnode;
    }
    total_elements++;
      printf("the element prepended: %d \n",data);
}

void append(int data)
{
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;
    newnode->next = newnode->prev = NULL;
    if(head == NULL)     {
        head = newnode;
        tail = newnode;
    }
```

```
   else
   {
      temp = head;
      while(temp->next != NULL) {
         temp = temp->next;
      }
      temp->next = newnode;
      newnode->prev = temp;
      tail = newnode;
   }
   total_elements++;
   printf("the element appended: %d \n",data);
}
void display()
{
   temp = head;
   printf("\n NULL <->");
   while(temp != NULL)    {
      printf("%d <->",temp->data);
      temp = temp->next;
   }
   printf("NULL \n");
}
void reverse() {
   temp = tail;
   printf("\n NULL <->");
   while(temp != NULL)    {
      printf("%d <->",temp->data);
      temp = temp->prev;
   }
   printf("NULL\n");
}
int main() {
   append(10);
   append(20);
   append(30);
   display();
   prepend(40);
   display();
   printf("\n total elements:%d",total_elements);
   printf("\n----\n");
   delete(30);
   display();
   printf("\n total elements:%d",total_elements);
         reverse();
   return 0;
}
```

Output:

```
the element appended: 10
the element appended: 20
the element appended: 30

 NULL <->10 <->20 <->30 <->NULL
the element prepended: 40

 NULL <->40 <->10 <->20 <->30 <->NULL

 total elements:4
  ----
the element deleted: 30

 NULL <->40 <->10 <->20 <->NULL

 total elements:3
 NULL <->20 <->10 <->40 <->NULL
```

Result:

Thus, the Doubly Linked List was successfully implemented in C and its ADT operations successfully executed successfully.

Experiment:4 **Stack Implementation Using Array**

Aim:

To write and execute a C program to implement a **Stack ADT using an array** and perform **Push, Pop, and Display operations**

Algorithm:

1. Initialize Stack

1. Declare an array stack[100] to store stack elements.
2. Set the maximum stack size n = 5.
3. Initialize top = -1 to indicate that the stack is empty.

2. Push Operation

1. Receive the element x to be inserted.
2. Check whether the stack is full:
3. if top >= n-1
   Display "Stack Overflow"
   Otherwise:
   top = top + 1
   stack[top] = x
4. Display the pushed element.

3. Pop Operation

1. Check whether the stack is empty:
2. if top <= -1
   Display "Stack Underflow"
   Otherwise:
   Display stack[top]
   stack[top] = 0
   top = top - 1
3. Thus, the topmost element is removed from the stack.

4. Display Operation

1. Check whether top >= 0.
2. If the stack is not empty, traverse from top to 0.
3. Display each stack element.
4. If top < 0, display **Stack is empty**.

   Start from TOP
   ↓
   Display stack[top]
   ↓
   top = top - 1
   ↓
   Repeat until top < 0

Program:

```
#include<stdio.h>

int stack[100], n, top, i;

void push(int x)
{
   if(top >= n-1)
   {
      printf("\n STACK is over flow");
   }
   else
   {
      top++;
      stack[top] = x;
      printf("\n element pushed into stack:%d",x);
   }
}

void pop()
{
   if(top <= -1)
   {
      printf("\n Stack is under flow");
   }
   else
   {
      printf("\n The popped elements is %d",stack[top]);
      stack[top] = 0;
      top--;
   }
}
void display()
{
   if(top >= 0)
   {
      printf("\n The elements in STACK:\n");

      for(i=top; i>=0; i--)
         printf("%d \n",stack[i]);
   }
   else
   {
      printf("\n The STACK is empty");
   }
}
```

```
int main()
{
   top = -1;
   n = 5;
   push(10);
   push(20);
   push(30);
   push(40);
   push(50);
   push(60);
   display();
   pop();
   pop();
   pop();
   pop();
   pop();
   pop();
   pop();
   return 0;
}
```

Output:

```
 element pushed into stack:10
 element pushed into stack:20
 element pushed into stack:30
 element pushed into stack:40
 element pushed into stack:50
 STACK is over flow
 The elements in STACK:
50
40
30
20
10
 The popped elements is 50
 The popped elements is 40
 The popped elements is 30
 The popped elements is 20
 The popped elements is 10
 Stack is under flow
```

Result:

Thus, the Stack ADT was successfully implemented using an array in C, and the push, pop, display operations were successfully demonstrated.

Experiment:5 **Stack Using Linked List**

Aim:
To write and execute a C program to implement a **Stack using a Linked List** with dynamic memory allocation and perform **Push, Pop, and Display operations**.

Algorithm:
1. Initialize Stack

1. Define a node containing data and next.
2. Initialize top = NULL to represent an empty stack.

2. Push Operation

1. Create a new node dynamically using malloc().
2. Store the given data in newNode->data.
3. Set newNode->next = NULL.
4. If the stack is empty:
   top = newNode
   Otherwise:
   newNode->next = top
   top = newNode
5. Display the pushed element.

3. Pop Operation

1. Check whether top == NULL.
2. If true, display **EMPTY STACK**.
3. Otherwise:
   temp = top
   temp_data = top->data
   top = top->next
4. Return the data of the deleted node.

4. Display Operation

1. Check whether top == NULL.
2. If true, display **Stack Underflow**.
3. Otherwise, set temp = top.
4. Traverse the stack until temp != NULL:
   Display temp->data
   temp = temp->next
5. Display NULL after reaching the end.

Program:

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
   int data;
   struct node *next;
};

struct node *top = NULL, *temp, *newNode;

void push(int data)
{
   newNode = (struct node*)malloc(sizeof(struct node));

   newNode->data = data;
   newNode->next = NULL;

   if(top == NULL)
   {
      top = newNode;
   }
   else
   {
      newNode->next = top;
      top = newNode;
   }

   printf("element pushed:%d\n",data);
}

int pop()
{
   if(top == NULL)
   {
      printf("\nEMPTY STACK");
   }
   else
   {
      temp = top;
      int temp_data = top->data;
      top = top->next;
      free(temp);
      return temp_data;
   }
```

```
}

void display()
{
   if(top == NULL)
   {
      printf("\nStack Underflow\n");
   } else  {
      printf("\nThe stack is:\n");
      temp = top;
      while (temp != NULL)
      {
         printf("%d--->",temp->data);
         temp = temp->next;
      }
      printf("NULL\n\n");
   }
}

int main()
{
   push(10);
   push(20);
   push(30);

   display();

   printf("\nelement poped %d",pop());
   printf("\nelement poped %d",pop());
  display();
   return 0;
}
```

Output:

```
element pushed:10
element pushed:20
element pushed:30

The stack is:
30--->20--->10--->NULL

element poped 30
element poped 20

The stack is:
10--->NULL
```

Result:

Thus, the Stack was successfully implemented using a Linked List in C. The Push, Pop, and Display operations were successfully performed.

Experiment: 6 Queue Using Array

Aim

To write and execute a C program to implement a **Queue using an Array** and perform **Enqueue, Dequeue, and Display operations**.

Algorithm:

1. Initialize Queue

1. Declare an array queue[MAX] with maximum size 50.
2. Initialize front = -1 and rear = -1 to indicate an empty queue.

2. Enqueue Operation

1. Check whether the queue is full:
2. if rear == MAX-1
   Display "Queue Overflow"
   If the queue is initially empty:
   set: front = 0
   Increment rear: rear = rear + 1
   Store the given element: queue[rear] = data
3. Display the inserted element.

3. Dequeue Operation

1. Check whether the queue is empty:
2. if front == -1 OR front > rear:
   Display "Queue Underflow"
   Otherwise:
   display the element at queue[front].
   Increment front:
   front = front + 1
   The next element becomes the front element.

4. Display Operation

1. Check whether front == -1.
2. If true:
   display **Queue is empty**.
   Otherwise:
   start from front and traverse up to rear:
3. Repeat for i = front to rear
   Display queue[i]
4. Stop after displaying the element at rear.

Program:

```
#include<stdio.h>
#define MAX 50
int queue[MAX];

int rear = -1;
int front = -1;

void enqueue(int data)
{
   if(rear == MAX - 1)
      printf("Queue Overflow \n");
   else
   {
      if(front == -1)
      {
         front = 0;
      }

      rear = rear + 1;
      queue[rear] = data;

      printf("Element inserted is : %d\n", queue[rear]);
   }
}

void dequeue()
{
   if(front == -1 || front > rear)
   {
      printf("Queue Underflow \n");
      return;
   }
   else
   {
      printf("\nElement deleted is : %d\n", queue[front]);
      front = front + 1;
   }
}

void display()
{
   int i;

   if(front == -1)
```

```
        printf("Queue is empty \n");
    else
    {
        printf("Queue is : \n");

        for(i = front; i <= rear; i++)
            printf("%d ", queue[i]);
    }
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(50);
    enqueue(50);
    enqueue(60);

    display();

    dequeue();
    dequeue();

    display();

    return 0;
}
```

Output:

```
Element inserted is : 10
Element inserted is : 20
Element inserted is : 30
Element inserted is : 50
Element inserted is : 50
Element inserted is : 60
Queue is :
10 20 30 50 50 60
Element deleted is : 10
Element deleted is : 20

Queue is :
30 50 50 60
```

Result:

Thus, the Queue was successfully implemented using an Array in C, and the Enqueue, Dequeue, and Display operations were successfully performed.

Experiment: 7 **Queue Using Linked List**

Aim:

To write and execute a C program to implement a Queue using a Linked List and perform Enqueue, Dequeue, and Display operations.

Algorithm:

1. Initialize Queue

1. Define a node containing data and next.
2. Initialize both front and rear to NULL.
3. front points to the first node and rear points to the last node.

2. Enqueue Operation

1. Create a new node dynamically using malloc().
2. Store the given value in ptr->data.
3. Set ptr->next = NULL.
4. If the queue is empty:
   front = rear = ptr
   Otherwise:
   rear->next = ptr
   rear = ptr
5. Display the inserted element.

3. Dequeue Operation

1. Check whether front == NULL.
2. If true:
   display **Underflow** and return -1.
   Otherwise:
   temp = front
   temp_data = front->data
   front = front->next
3. Return the deleted element.

4. Display Operation

1. Check whether both front and rear are NULL.
2. If true: display Queue is Empty
   Otherwise:
   set temp = front.
3. Traverse the queue:
   Repeat until temp != NULL:
   Display temp->data

temp = temp->next

4. Display NULL after reaching the end.

Program:

```
#include<stdio.h>
#include<stdlib.h>
/* Structure to create a node with data and next pointer */
struct node
{
   int data;
   struct node *next;
};

struct node *front = NULL;
struct node *rear = NULL;

/* Enqueue operation on a queue */

void enqueue(int value)
{
   struct node *ptr;

   ptr = (struct node*)malloc(sizeof(struct node));
   ptr->data = value;
   ptr->next = NULL;

   if((front == NULL) && (rear == NULL))
   {
      front = rear = ptr;
   }
   else
   {
      rear->next = ptr;
      rear = ptr;
   }
   printf("Node is Inserted:%d\n\n",value);
}

/* Dequeue operation on a queue */
int dequeue()
{
   if(front == NULL)
   {
      printf("\nUnderflow\n");
      return -1;
   }
   else
   {
```

```
        struct node *temp = front;
        int temp_data = front->data;

        front = front->next;

        free(temp);

        return temp_data;
    }
}

/* Display all elements of queue */
void display()
{
    struct node *temp;
    if((front == NULL) && (rear == NULL))
    {
        printf("\nQueue is Empty\n");
    }
    else
    {
        printf("The queue is \n");
        temp = front;
        while(temp)
        {
            printf("%d--->",temp->data);
            temp = temp->next;
        }
        printf("NULL\n\n");
    }
}
int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    return 0;
}
```

Output:

Node is Inserted:10

Node is Inserted:20

Node is Inserted:30

The queue is
10--->20--->30--->NULL

The queue is
20--->30--->NULL

Result:

Thus, the Queue was successfully implemented using a Linked List in C. The Enqueue, Dequeue, and Display operations were successfully performed.

Experiment: 8 **Circular Queue Using Array**

Aim:

To write and execute a C program to implement a Circular Queue using an Array and perform Enqueue, Dequeue, and Display operations.

Algorithm:

1. Initialize Circular Queue

1. Declare an array items[SIZE] with SIZE = 5.
2. Initialize:
   front = -1
   rear = -1
3. front represents the first element and rear represents the last element.

2. Check Queue is Full

1. The circular queue is full when:
   If (rear + 1) % SIZE == front
   Return 1 if full;
   Otherwise:
   return 0.

3. Check Queue is Empty

1. If front == -1, the queue is empty.
   Return 1;
   Otherwise:
   return 0.

4. Enqueue

1. Check whether the queue is Full.
2. If isFull:
   display **Queue is full**.
3. Otherwise:
   if the queue is empty:
   front = 0
   rear = 0
   Otherwise, move rear circularly:
   rear = (rear + 1) % SIZE
   Store the element at items[rear].
4. Display the inserted element.

5. Dequeue

1. Check whether the queue is Empty.
2. If is Empty:
   display **Queue is empty** and return -1.
   Otherwise:
   store items[front] in element.

If front == rear:
the queue contains only one element:
front = -1
rear = -1
Otherwise, move front circularly:
front = (front + 1) % SIZE

3. Display and return the deleted element.

6. Display

1. Check whether the queue is Empty.
2. If **is Empty**:
   display **Empty Queue**.
   Otherwise:
   start from front.
   Display each element and move circularly:
   i = (i + 1) % SIZE
   Continue until i == rear.
3. Display the front and rear positions.

Program:

```
#include <stdio.h>
#define SIZE 5
int items[SIZE];
int front = -1, rear = -1;
int isFull()
{
   if ((front == rear + 1) ||   (front == 0 && rear == SIZE - 1))
      return 1;
   return 0;
}
int isEmpty()
{
   if (front == -1)
      return 1;
   return 0;
}

void enQueue(int element)
{
   if (isFull())
   {
      printf("\nQueue is full!!\n");
   }
   else
   {
      if (front == -1)  {
         front = 0;
```

```
            rear = 0;
        } else {
            rear = (rear + 1) % SIZE;
        }
        items[rear] = element;
        printf("\nInserted -> %d", element);
    }
}
int deQueue()
{
    int element;
    if (isEmpty())
    {
        printf("\nQueue is empty!!\n");
        return -1;
    }   else   {
                element = items[front];
                if (front == rear)
                {
                    front = -1;
                    rear = -1;
                }  else     {
                    front = (front + 1) % SIZE;
                }
                printf("\nDeleted element -> %d\n", element);
        return element;
    }
}
void display()
{
    int i;
    if (isEmpty())
    {
        printf("\nEmpty Queue\n");
    }
    else
    {
        printf("\nFront -> %d", front);
        printf("\nItems -> ");
        for (i = front; i != rear; i = (i + 1) % SIZE)
        {
            printf("%d ", items[i]);
        }
        printf("%d ", items[i]);
        printf("\nRear -> %d\n", rear);
    }
}
```

```
int main()
{
   deQueue();
   enQueue(1);
   enQueue(2);
   enQueue(3);
   enQueue(4);
   enQueue(5);
   enQueue(6);
   display();
   deQueue();
   display();
   enQueue(7);
   display();
   enQueue(8);
   return 0;
}
```

Output:

```
Queue is empty!!

Inserted -> 1
Inserted -> 2
Inserted -> 3
Inserted -> 4
Inserted -> 5
Queue is full!!

Front -> 0
Items -> 1 2 3 4 5
Rear -> 4
Deleted element -> 1
Front -> 1
Items -> 2 3 4 5
Rear -> 4
Inserted -> 7
Front -> 1
Items -> 2 3 4 5 7
Rear -> 0
Queue is full!!
```

Result:

Thus, the Circular Queue was successfully implemented using an array in C. The Enqueue, Dequeue, Display, Overflow, Underflow, operations were successfully executed.